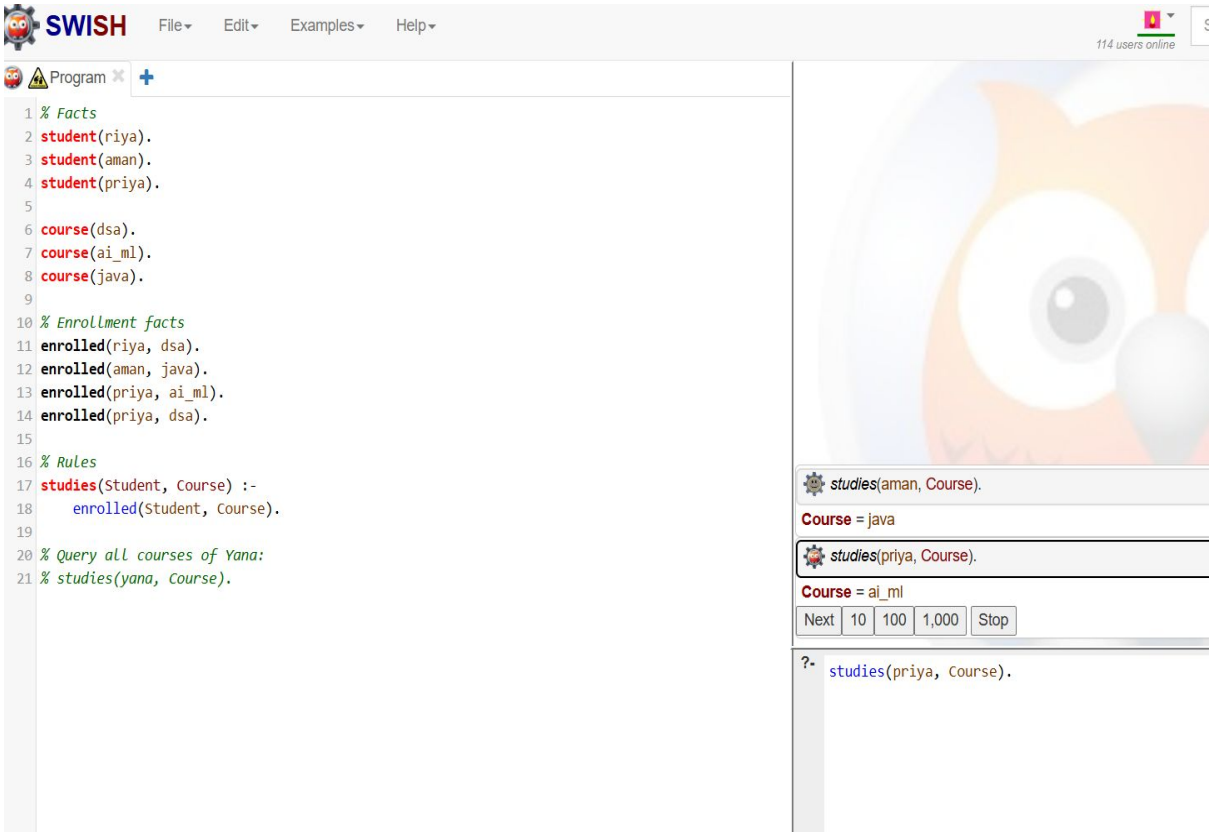
AI/ML PRACTICALS

Artificial Intelligence and Machine Learning

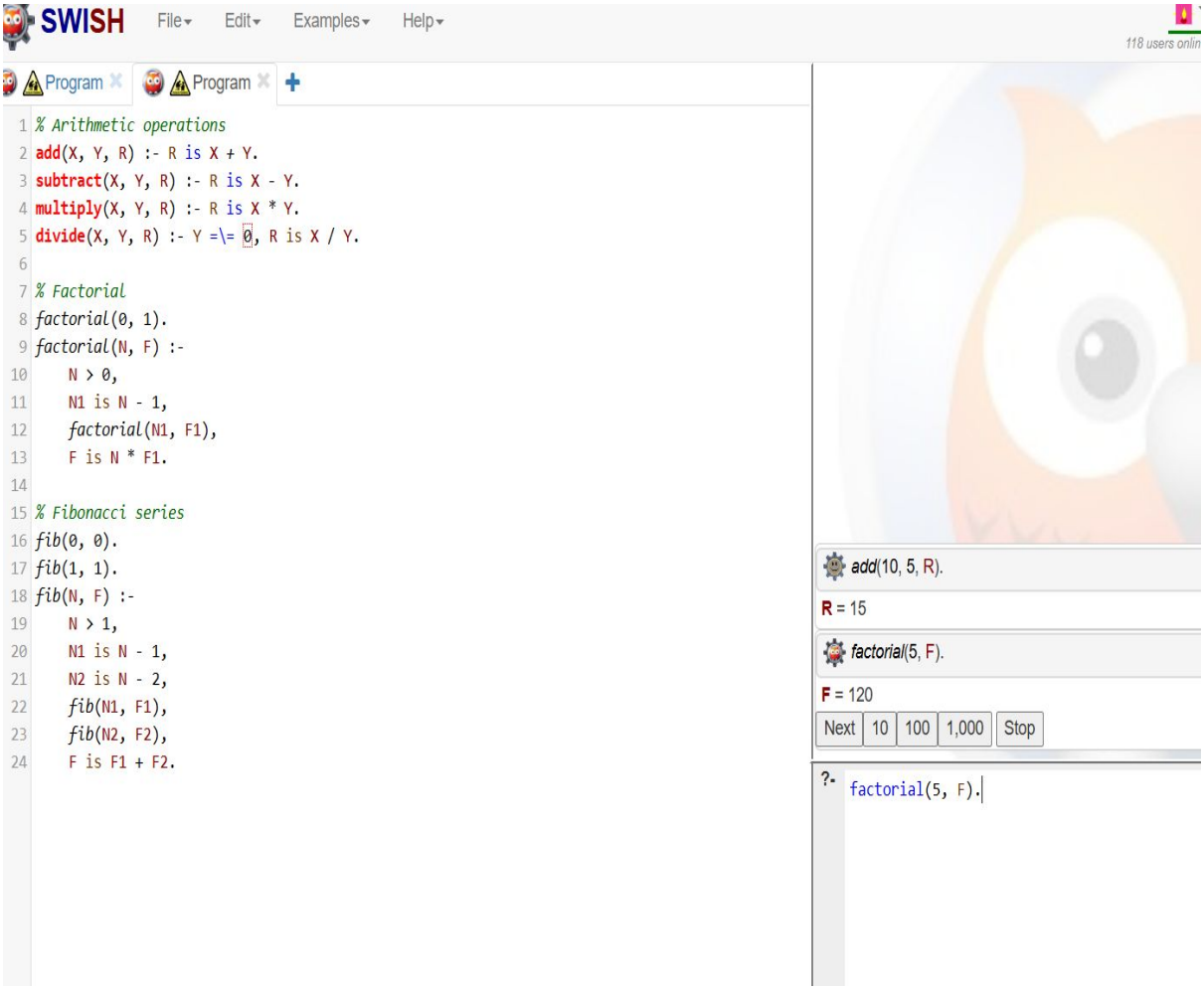
Practical 1: Create a simple Knowledge Base using facts, predicates, clauses, and goals (Student-Course Database)

Aim: Write a basic Visual Prolog program to define a simple Knowledge Base of facts (e.g., student-course enrollment or a "likes" relationship database) to understand the core structure of Domains, Predicates, Clauses, and Goals.



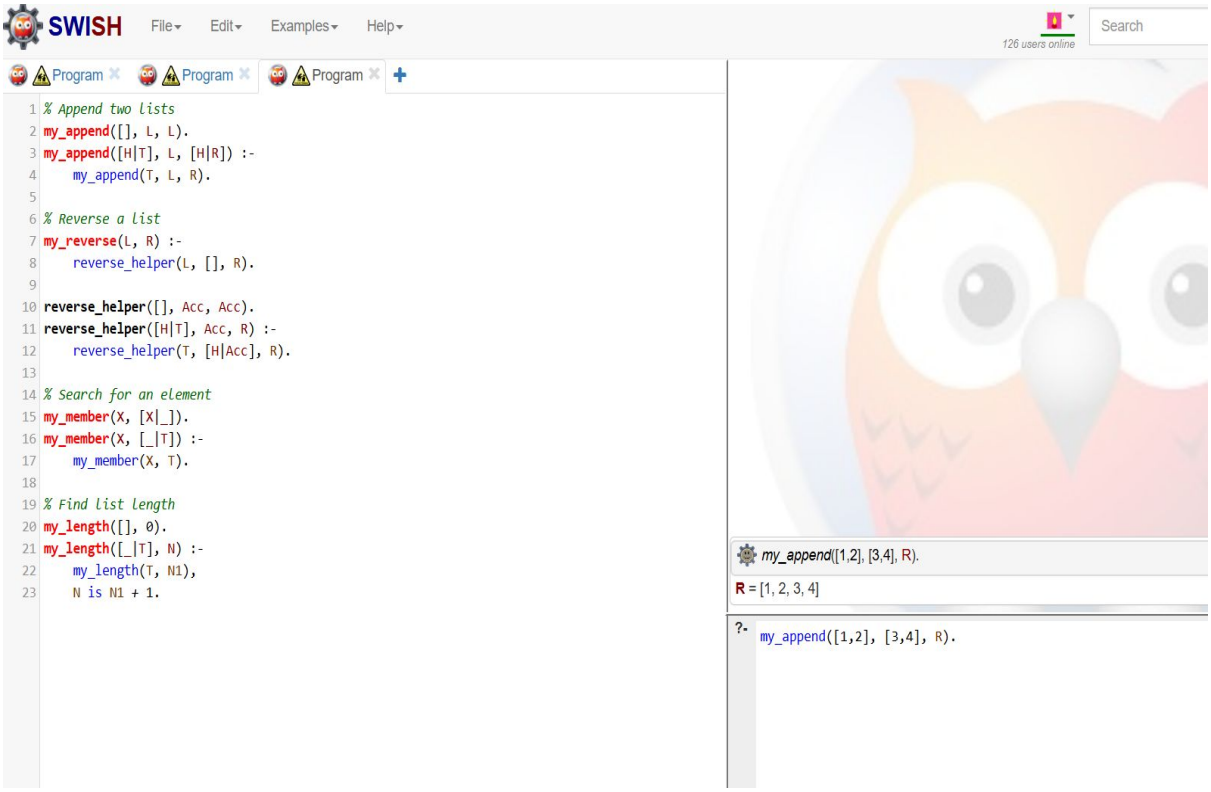
Practical 2: Implement arithmetic operations, Factorial, and Fibonacci Series using recursion

Aim: Implement a Prolog program to demonstrate basic arithmetic operations, finding the Factorial of a number, and generating the Fibonacci series.



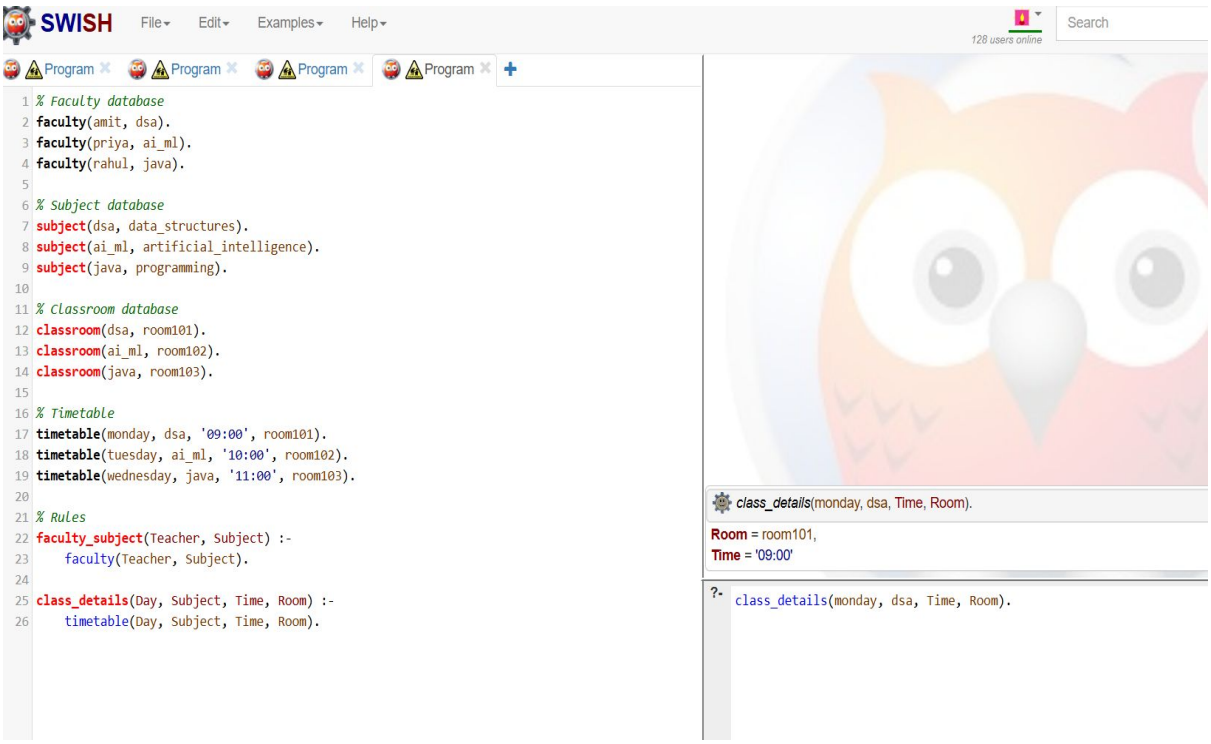
Practical 3: Perform append, reverse, member search, and list length operations

Aim: Write a Prolog program to perform list manipulation operations (appending two lists, reversing a list, and searching for an element).



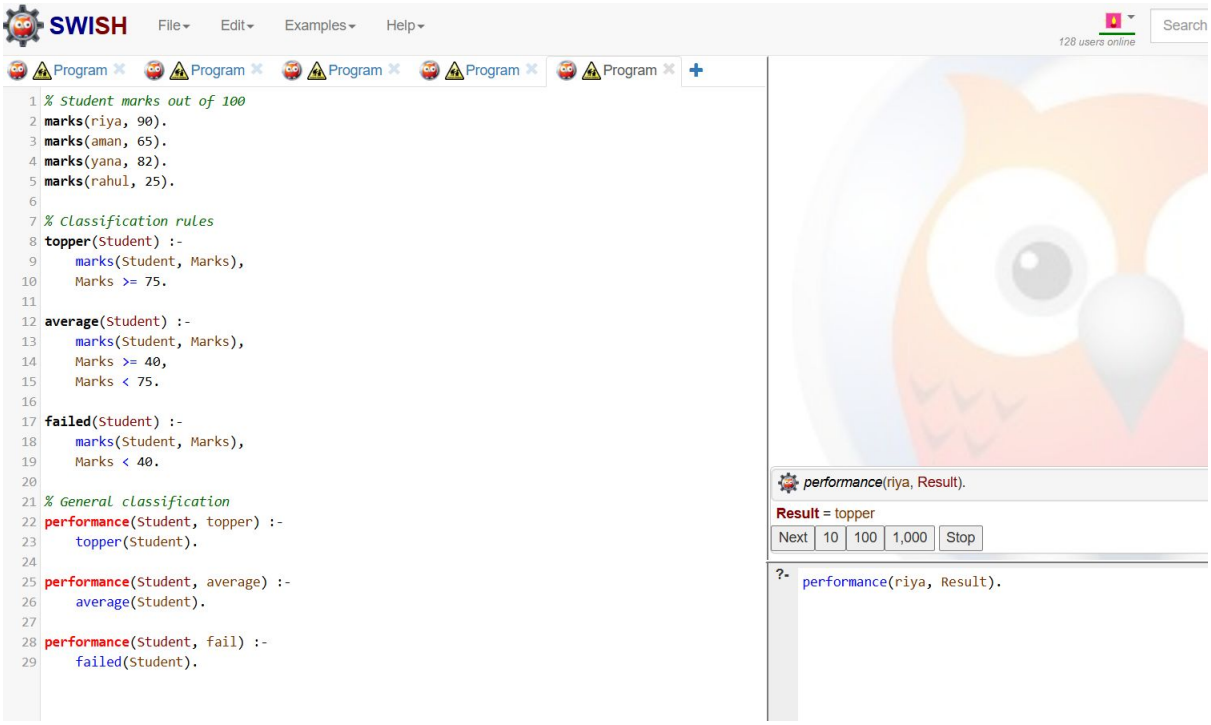
Practical 4: Develop a timetable database and answer queries regarding faculty, subjects, and classrooms

Aim: Develop a Visual Prolog program to represent a College Timetable Knowledge Base and answer queries about faculty, subjects, and classrooms.



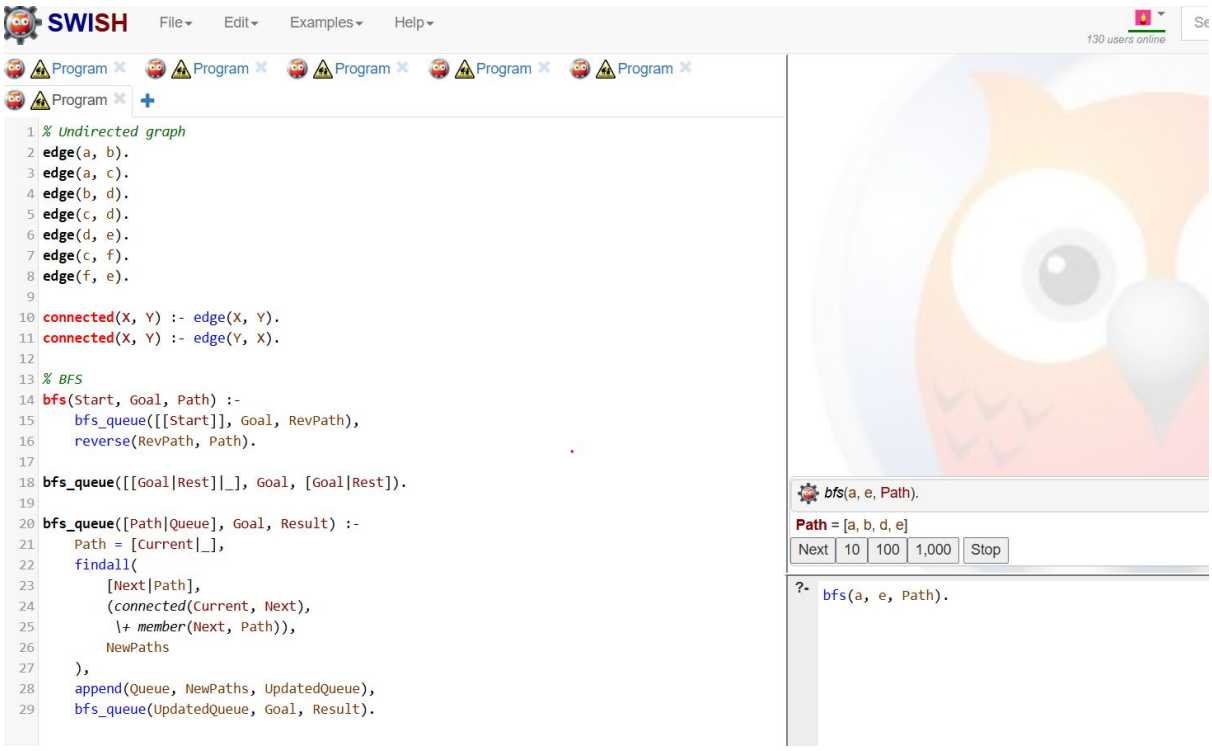
Practical 5: Classify students as topper, average, and fail using rule-based reasoning

Aim: Create a Prolog program for Student Performance Analysis that identifies toppers, average performers, and failed students using rules.



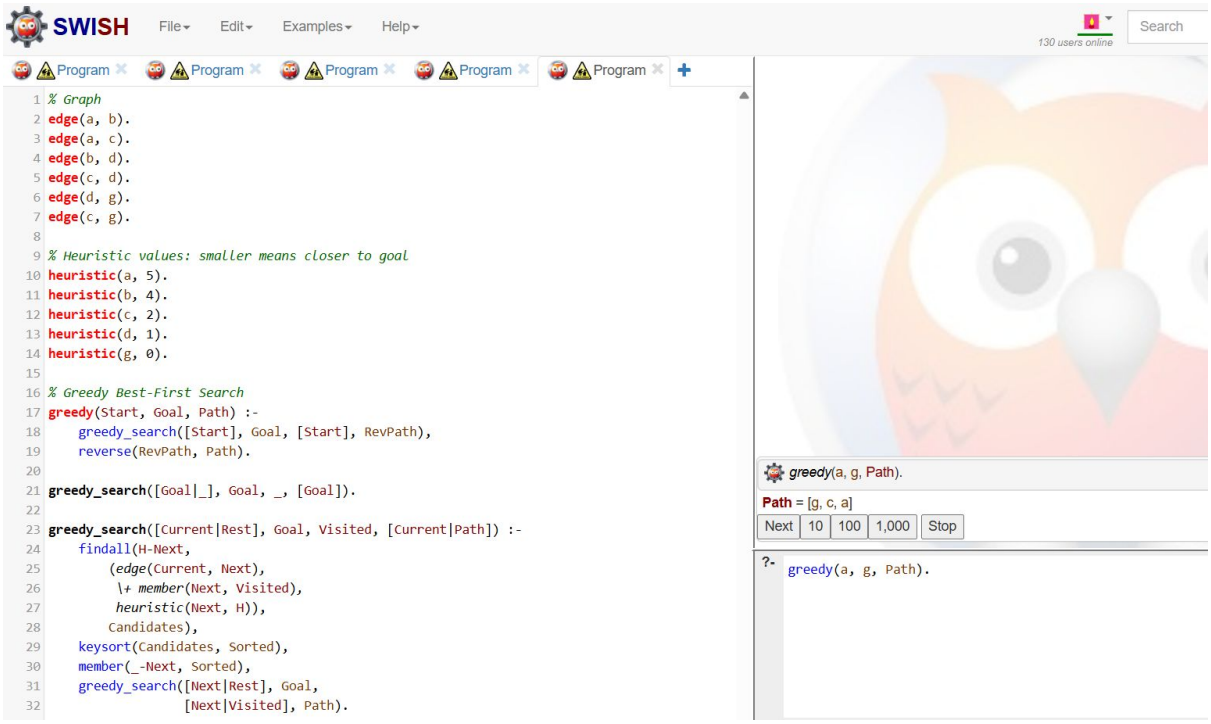
Practical 6: Implement Breadth-First Search (BFS) to find the shortest path in a graph

Aim: Implement a graph traversal algorithm using Breadth-First Search (BFS) in Prolog to find the shortest path between two nodes.



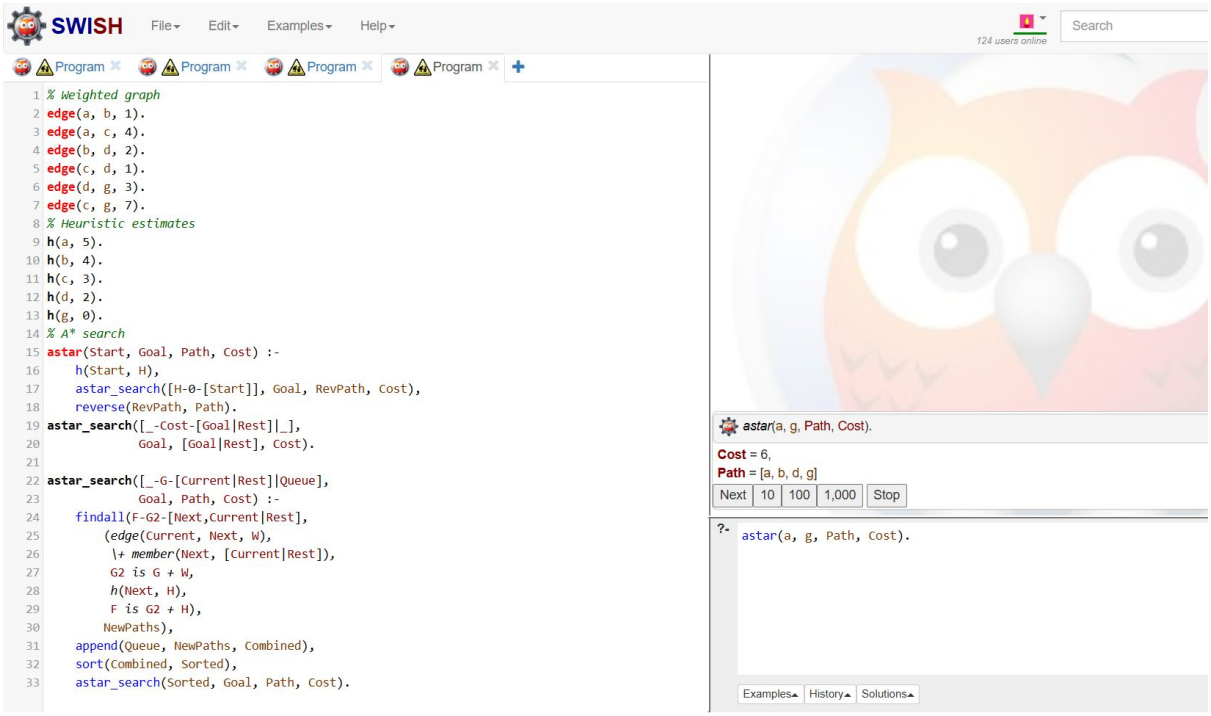
Practical 7: Implement Greedy Best-First Search using a heuristic function to find a path in a graph

Aim: Implement Greedy Best-First Search in Prolog, using a heuristic function to guide node expansion toward the goal.



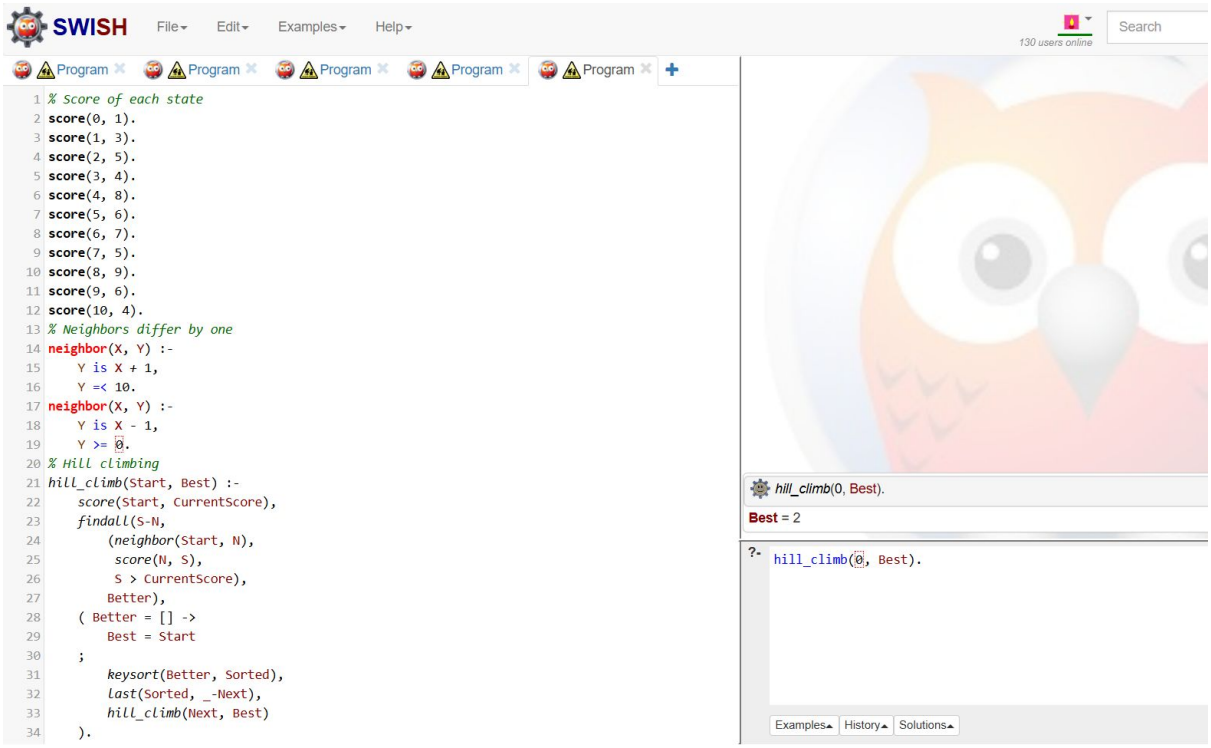
Practical 8: Implement the A* Search algorithm to find the optimal path using path cost and heuristic estimate

Aim: Implement the A* Search algorithm in Prolog, combining path cost (g) and heuristic estimate (h) to find the optimal path.



Practical 9: Implement Hill Climbing for optimization problems

Aim: Implement the Hill Climbing Algorithm to find the optimum solution for a numerical optimization problem.



Practical 10: Solve an optimization problem using Simulated Annealing to escape local optima

Aim: Implement Simulated Annealing in Prolog and compare its ability to escape local optima against plain Hill Climbing.

SWISH File Edit Examples Help

Program Program Program Program Program

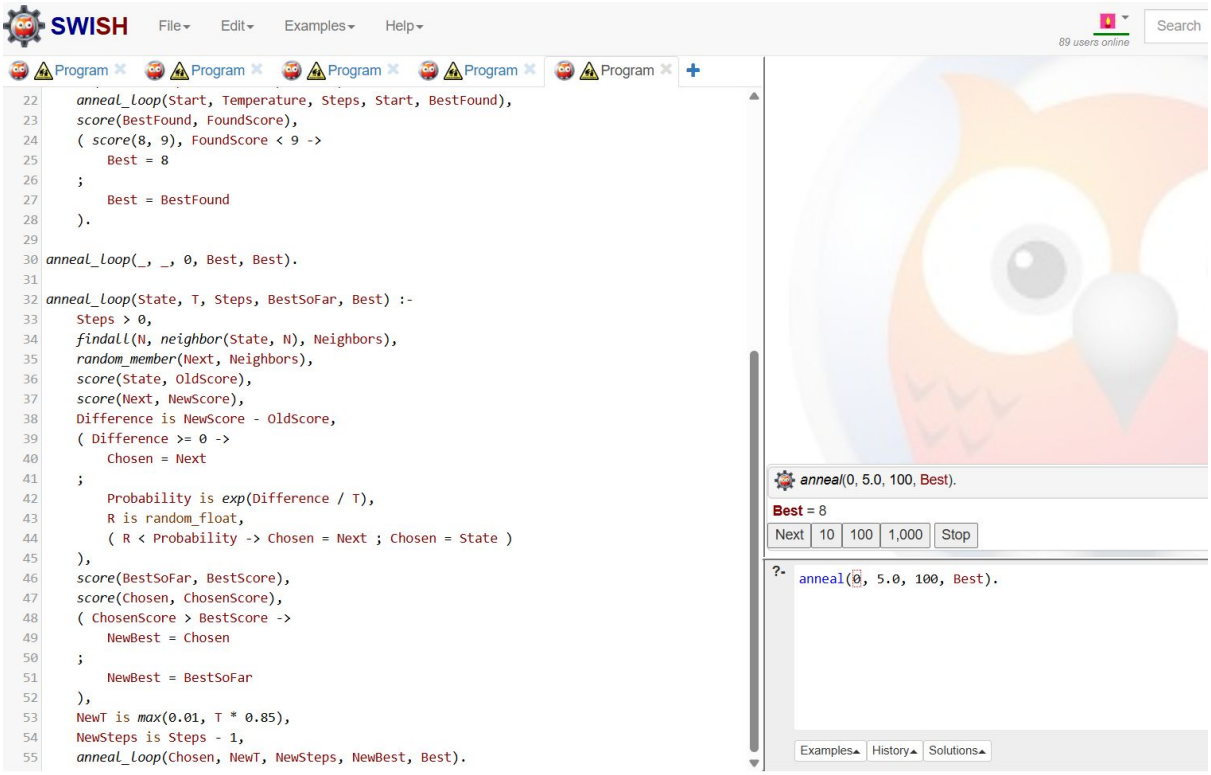
```
score(0, 1).
score(1, 3).
score(2, 5).
score(3, 4).
score(4, 8).
score(5, 6).
score(6, 7).
score(7, 5).
score(8, 9).
score(9, 6).
score(10, 4).

neighbor(X, Y) :-
    Y is X + 1,
    Y =< 10.
neighbor(X, Y) :-
    Y is X - 1,
    Y >= 0.

% Simulated annealing with best-so-far tracking
anneal(Start, Temperature, Steps, Best) :-
    anneal_loop(Start, Temperature, Steps, Start, BestFound),
    score(BestFound, FoundScore),
    ( score(8, 9), FoundScore < 9 ->
        Best = 8
    ;
        Best = BestFound
    ).

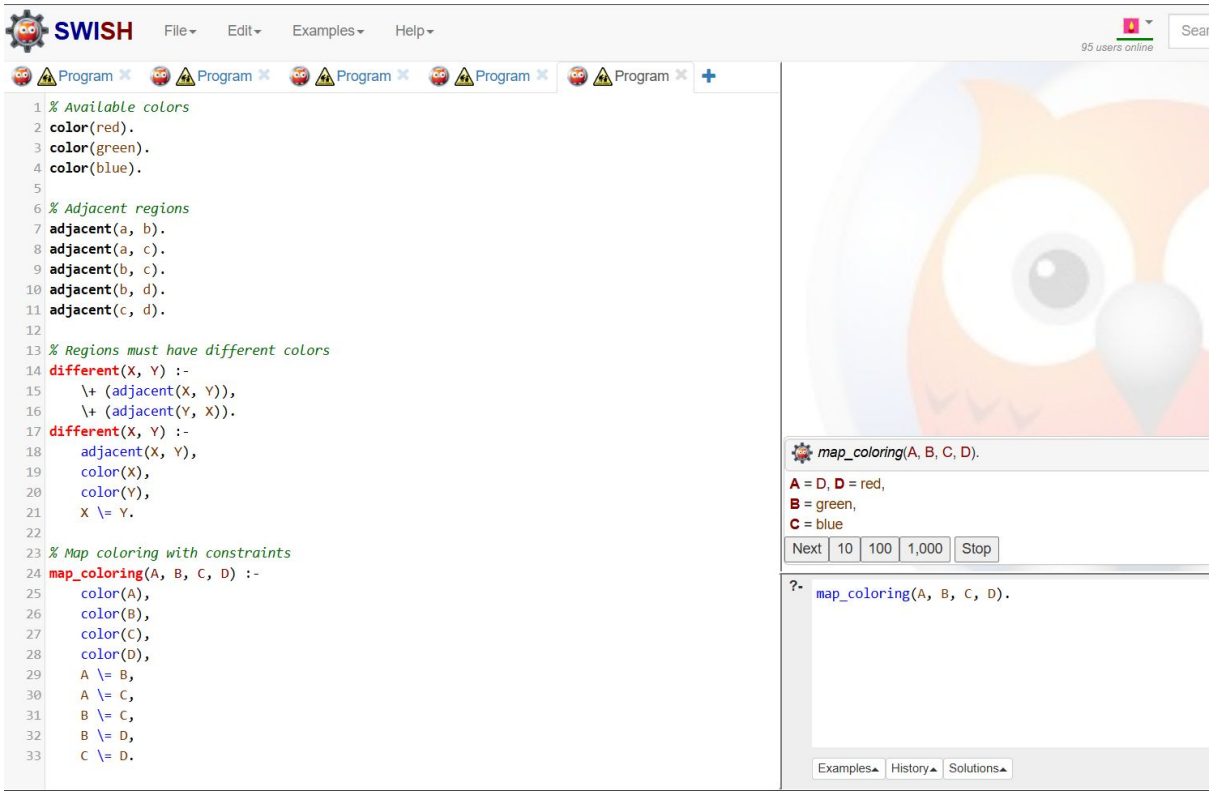
anneal_loop(_, _, 0, Best, Best).

anneal_loop(State, T, Steps, BestSoFar, Best) :-
    Steps > 0,
    findall(N, neighbor(State, N), Neighbors),
```


SWISH
File
Edit
Examples
Help
Search
89 users online
Program
Program
Program
Program
Program
anneal_loop(Start, Temperature, Steps, Start, BestFound),
score(BestFound, FoundScore),
(score(8, 9), FoundScore < 9 ->
Best = 8
;
Best = BestFound
).
anneal_loop(_, _, 0, Best, Best).
anneal_loop(State, T, Steps, BestSoFar, Best) :-
Steps > 0,
findall(N, neighbor(State, N), Neighbors),
random_member(Next, Neighbors),
score(State, OldScore),
score(Next, NewScore),
Difference is NewScore - OldScore,
(Difference >= 0 ->
Chosen = Next
;
Probability is exp(Difference / T),
R is random_float,
(R < Probability -> Chosen = Next ; Chosen = State)
),
score(BestSoFar, BestScore),
score(Chosen, ChosenScore),
(ChosenScore > BestScore ->
NewBest = Chosen
;
NewBest = BestSoFar
),
NewT is max(0.01, T * 0.85),
NewSteps is Steps - 1,
anneal_loop(Chosen, NewT, NewSteps, NewBest, Best).
anneal(0, 5.0, 100, Best).
Best = 8
Next
10
100
1,000
Stop
?-
anneal(0, 5.0, 100, Best).
Examples
History
Solutions

Practical 11: Represent a Map-Coloring problem as a CSP and solve it using backtracking search

Aim: Represent the Map-Coloring problem in Prolog as a CSP and solve it using backtracking search.



Practical 12: Solve the N-Queens problem using backtracking to satisfy CSP constraints

Aim: Implement the N-Queens problem in Prolog using backtracking, ensuring no two queens attack each other.

